

Bayesian Survival Analysis: A Donner Party Case Study

Jayking Wu

Introduction

The Donner Party tragedy of 1846 provides a historical case for studying survival under extreme environmental conditions. After becoming trapped in the Sierra Nevada mountains during winter, many members of the group died before rescue arrived. This report examines whether age and gender were associated with survival outcomes among the 45 individuals included in the Donner dataset. The main research question is whether younger individuals and females had higher probabilities of survival.

Data Description

The Donner dataset contains 45 observations on individuals from the Donner Party. Three variables are used in the analysis: age (measured in years), male (1 = male, 0 = female), and survival (1 = survived, 0 = died). The average age in the sample was 31.8 years, with ages ranging from 15 to 65. Approximately two-thirds of the sample were male.

Exploratory Data Analysis

As shown in Figure 1, female members had a survival rate of 66.7%, compared with 33.3% for males. Figure 2 shows that survivors were younger on average, with a mean age of 27.2 years compared with 35.5 years for non-survivors.

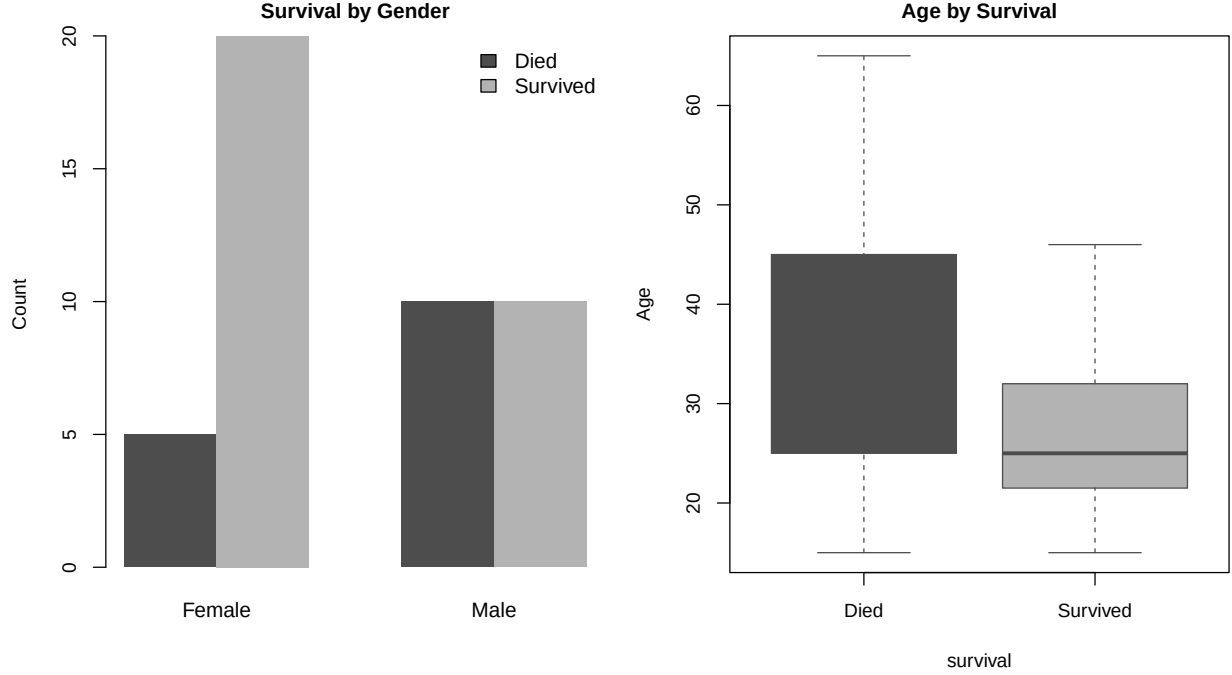


Figure 1: Exploratory plots of survival by gender and age.

Several limitations should be noted before formal modeling. First, the sample size is relatively small, which increases uncertainty in parameter estimates. Second, the dataset includes only age and gender, while other potentially important factors such as family support, health condition, food access, and travel position are omitted. Finally, the sample may not fully represent all original Donner Party members, which may introduce selection bias.

Bayesian Model and Prior Assumptions

For each individual i , let $y_i = 1$ if the individual survived and $y_i = 0$ otherwise. Conditional on the survival probability p_i , I model

$$y_i \mid p_i \sim \text{Bernoulli}(p_i).$$

The linear predictor is defined as

$$\eta_i = \beta_0 + \beta_1 \text{male}_i + \beta_2 \text{age}_i,$$

where β_0 is the intercept, β_1 represents the effect of gender, and β_2 represents the effect of age.

Three Bayesian binary regression models are considered, which differ only in the link function:

$$p_i = \begin{cases} \Phi(\eta_i), & \text{probit} \\ \frac{\exp(\eta_i)}{1 + \exp(\eta_i)}, & \text{logit} \\ 1 - \exp\{-\exp(\eta_i)\}, & \text{complementary log-log} \end{cases}$$

Independent priors are assigned to the regression coefficients. The priors are centered at zero, reflecting the assumption that, before observing the data, there is no strong prior belief about the direction of the effects.

To account for differences in the scale of the covariates, a smaller prior variance is used for the coefficient of age. The variable *male* takes values 0 or 1, whereas *age* ranges from approximately 15 to 65, so even a small age coefficient can substantially affect the linear predictor.

Specifically,

$$\beta_0 \sim N(0, 10^2), \quad \beta_1 \sim N(0, 10^2), \quad \beta_2 \sim N(0, 0.1^2).$$

This specification provides weakly informative priors while preventing unrealistically large effects for the age coefficient.

Model Fitting

Posterior inference was carried out using a Markov chain Monte Carlo (MCMC) method, specifically a random-walk Metropolis algorithm implemented in R. This approach was used because the posterior distribution does not have a closed-form expression under the Bernoulli likelihood combined with the chosen link functions.

For each model, new parameter values were proposed by adding a small random perturbation to the current state. The proposed values were accepted or rejected based on their posterior probabilities, ensuring that the resulting Markov chain converges to the target posterior distribution.

All chains were initialized at $(\beta_0, \beta_1, \beta_2) = (0, 0, 0)$, which represents a neutral starting point. For each link function, I ran the algorithm for 55,000 iterations and discarded the first 15,000 iterations as burn-in to reduce the influence of initial values. The remaining 40,000 samples were used to approximate the posterior distributions of the regression coefficients.

The total number of iterations was chosen to ensure a sufficient number of effective samples after removing the burn-in period. The proposal scales were tuned separately for the probit, logit, and complementary log-log models to obtain reasonable acceptance rates and allow efficient exploration of the parameter space.

Table 1: Acceptance rates for the random-walk Metropolis samplers.

	Model	AcceptanceRate
probit	probit	0.505
logit	logit	0.479
cloglog	cloglog	0.498

Convergence Diagnostics

Convergence of the MCMC algorithm was assessed using trace plots, acceptance rates, and autocorrelation-based diagnostics.

Trace plots for all parameters fluctuated around stable central values with no visible long-term trends, suggesting that the Markov chains had reached their stationary distributions. In addition, the acceptance rates for all three models were within a reasonable range, indicating that the proposal distributions were appropriately tuned.

To further assess sampling efficiency, I examined the autocorrelation functions of the post-burn-in samples. The autocorrelation at lag k measures the dependence between MCMC draws that are k iterations apart. Ideally, autocorrelation should decrease rapidly as the lag increases.

The autocorrelation plots show noticeable dependence among posterior samples, especially for β_0 and β_2 . This behavior is common for random-walk Metropolis algorithms because proposed updates are local and can produce correlated samples.

This pattern is also reflected in the effective sample size. Although 40,000 post-burn-in draws were retained, the effective sample sizes are smaller than the raw number of samples. Therefore, the sampler is not perfectly efficient. However, the chains show stable behavior, and the main conclusions are consistent across the three link functions. Overall, the diagnostics suggest that the MCMC samples provide a reasonable approximation to the posterior distributions.

Table 2: Effective sample size estimates for post-burn-in posterior samples.

	Model	Parameter	ESS
beta0...1	Probit	beta0	343
beta__male...2	Probit	beta__male	632
beta__age...3	Probit	beta__age	391
beta0...4	Logit	beta0	334
beta__male...5	Logit	beta__male	647
beta__age...6	Logit	beta__age	383
beta0...7	C-log-log	beta0	350
beta__male...8	C-log-log	beta__male	721
beta__age...9	C-log-log	beta__age	385

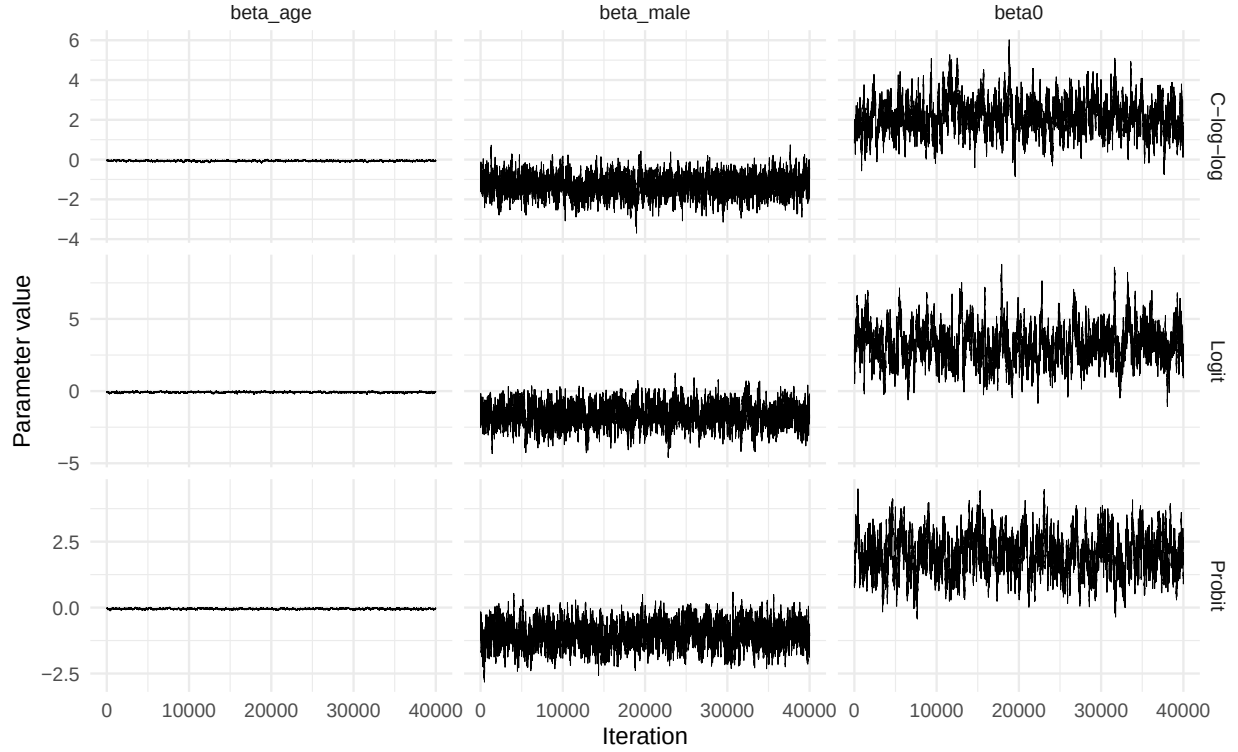


Figure 2: Trace plots for post-burn-in posterior samples.

Posterior Results

Table 1 summarizes the posterior means and 95% credible intervals for the regression coefficients under the three link functions.

Across all models, the posterior mean of β_1 is negative, indicating that males had lower estimated survival probabilities than females after adjusting for age. The posterior mean of β_2 is also negative, suggesting that survival probability decreases with age.

The 95% credible intervals for both coefficients do not include zero across the three models, which suggests that these effects are supported by the posterior distribution. The results are consistent across the probit, logit, and complementary log-log models, indicating that the conclusions are not sensitive to the choice of link function.

Table 3: Posterior summaries for regression coefficients.

	Model	Parameter	Mean	Lower	Upper
beta0...1	Probit	beta0	1.997	0.588	3.458
beta_male...2	Probit	beta_male	-1.011	-1.908	-0.157
beta_age...3	Probit	beta_age	-0.047	-0.088	-0.010
beta0...4	Logit	beta0	3.209	0.679	6.006
beta_male...5	Logit	beta_male	-1.665	-3.195	-0.194
beta_age...6	Logit	beta_age	-0.076	-0.153	-0.009

	Model	Parameter	Mean	Lower	Upper
beta0...7	C-log-log	beta0	2.155	0.557	3.933
beta_male...8	C-log-log	beta_male	-1.289	-2.306	-0.323
beta_age...9	C-log-log	beta_age	-0.064	-0.117	-0.019

Posterior Predictive Distribution

To assess model fit, I generated posterior predictive samples for the total number of survivors. For each posterior draw of the regression coefficients, I computed the survival probabilities for all individuals and simulated new survival outcomes from Bernoulli distributions. The total number of survivors was recorded for each simulated dataset.

Table 2 reports the posterior predictive mean and 95% intervals under each link function. The observed number of survivors in the dataset is 20.

The predictive means are close to the observed value, and the observed count lies within the 95% posterior predictive intervals for all three models. This suggests that the models provide a reasonable fit to the data. The results are consistent across the probit, logit, and complementary log-log specifications.

Table 4: Posterior predictive distribution for the number of survivors.

Model	Mean	Lower	Upper
Probit	20.20	12	28
Logit	19.93	12	28
C-log-log	19.59	12	28

Conclusion

This analysis suggests that both age and gender were associated with survival outcomes among members of the Donner Party. Across all three Bayesian models, the posterior estimates indicate that males had lower survival probabilities than females, and that survival probability decreases with age.

The results are consistent across the probit, logit, and complementary log-log specifications. Posterior predictive checks further show that all models produce reasonable predictions for the total number of survivors, indicating an adequate fit to the observed data.

However, these findings should be interpreted with caution. The dataset is relatively small and includes only two predictors, and important factors such as family structure, health condition, and access to resources are not observed. Therefore, the results reflect associations rather than causal relationships.

Appendix

R Code

```
library(LearnBayes)
library(dplyr)
library(tidyr)
library(knitr)
library(ggplot2)

data(donner)

head(donner)
str(donner)
summary(donner)

# Exploratory Data Analysis (EDA)

par(mfrow = c(1, 2), mar = c(4, 4, 2, 1))

# Barplot
tab <- table(donner$male, donner$survival)

barplot(
  tab,
  beside = TRUE,
  col = c("#4D4D4D", "#B3B3B3"),
  border = NA,
  legend.text = c("Died", "Survived"),
  args.legend = list(x = "topright", bty = "n"),
  names.arg = c("Female", "Male"),
  main = "Survival by Gender",
  ylab = "Count",
  cex.main = 1,
  cex.axis = 0.9,
  cex.lab = 0.9
)

# Boxplot
boxplot(
  age ~ survival,
  data = donner,
  col = c("#4D4D4D", "#B3B3B3"),
  border = "gray30",
  names = c("Died", "Survived"),
  ylab = "Age",
  main = "Age by Survival",
```

```

    cex.main = 1,
    cex.axis = 0.9,
    cex.lab = 0.9
)

par(mfrow = c(1, 1))

# Data preprocessing for modeling
data(donner)

dat <- donner %>%
  mutate(
    y = as.numeric(survival),
    x_male = as.numeric(male),
    x_age = as.numeric(age)
  )

X <- model.matrix(~ x_male + x_age, data = dat)
y <- dat$y

# Prior specification

## Standard deviations for priors
prior_sd <- c(10, 10, 0.1)

# Link functions (inverse links)
inv_link <- list(
  probit = function(z) pnorm(z),
  logit = function(z) plogis(z),
  cloglog = function(z) 1 - exp(-exp(z))
)

# Log-posterior function factory

## Returns a function that computes log posterior

log_post_factory <- function(link_name) {
  link_fun <- inv_link[[link_name]]

  function(beta) {
    eta <- as.vector(X %*% beta)
    p <- link_fun(eta)
    p <- pmin(pmax(p, 1e-10), 1 - 1e-10)

    log_lik <- sum(y * log(p) + (1 - y) * log(1 - p))
  }
}

```

```

    log_pr <- sum(dnorm(beta, mean = 0, sd = prior_sd, log = TRUE))

    log_lik + log_pr
  }
}

# Random-walk Metropolis algorithm
rw_metropolis <- function(logpost, start, proposal_sd, n_iter) {
  beta_current <- start
  lp_current <- logpost(beta_current)

  draws <- matrix(NA, nrow = n_iter, ncol = length(start))
  colnames(draws) <- c("beta0", "beta_male", "beta_age")

  accepted <- 0

  for (t in seq_len(n_iter)) {
    beta_proposed <- beta_current + rnorm(length(start), mean = 0, sd = proposal_sd)
    lp_proposed <- logpost(beta_proposed)

    log_accept_ratio <- lp_proposed - lp_current

    if (log(runif(1)) < log_accept_ratio) {
      beta_current <- beta_proposed
      lp_current <- lp_proposed
      accepted <- accepted + 1
    }

    draws[t, ] <- beta_current
  }

  list(
    draws = draws,
    accept_rate = accepted / n_iter
  )
}

# Run MCMC

set.seed(3303)

n_iter <- 55000
burn_in <- 15000
start_value <- c(0, 0, 0)

```

```

## Proposal standard deviations (tuned separately)
proposal_sd <- list(
  probit = c(0.20, 0.20, 0.008),
  logit = c(0.35, 0.35, 0.015),
  cloglog = c(0.25, 0.25, 0.010)
)

mcmc_fit <- list()

## Run MCMC for each model
for (link_name in names(proposal_sd)) {
  mcmc_fit[[link_name]] <- rw_metroplis(
    logpost = log_post_factory(link_name),
    start = start_value,
    proposal_sd = proposal_sd[[link_name]],
    n_iter = n_iter
  )
}

posterior_draws <- lapply(
  mcmc_fit,
  function(fit) fit$draws[(burn_in + 1):n_iter, ]
)

acceptance_table <- data.frame(
  Model = names(mcmc_fit),
  AcceptanceRate = sapply(mcmc_fit, function(fit) fit$accept_rate)
)

kable(
  acceptance_table,
  digits = 3,
  caption = "Acceptance rates for the random-walk Metropolis samplers."
)

# Effective Sample Size (ESS)
estimate_ess <- function(x, max_lag = 100) {
  n <- length(x)
  acf_values <- acf(x, plot = FALSE, lag.max = max_lag)$acf
  acf_values <- as.numeric(acf_values)[-1]
  n / (1 + 2 * sum(acf_values)) # ESS formula
}

make_ess_table <- function(samples, model_name) {
  data.frame(
    Model = model_name,

```

```

    Parameter = colnames(samples),
    ESS = sapply(as.data.frame(samples), estimate_ess)
  )
}

## Combine ESS results
ess_table <- bind_rows(
  make_ess_table(posterior_draws$probit, "Probit"),
  make_ess_table(posterior_draws$logit, "Logit"),
  make_ess_table(posterior_draws$cloglog, "C-log-log")
) %>%
  mutate(ESS = round(ESS, 0))

kable(
  ess_table,
  caption = "Effective sample size estimates for post-burn-in posterior samples."
)

# Trace plots

## Reshape data for plotting
trace_data <- bind_rows(
  as.data.frame(posterior_draws$probit) %>%
    mutate(Iteration = row_number(), Model = "Probit"),
  as.data.frame(posterior_draws$logit) %>%
    mutate(Iteration = row_number(), Model = "Logit"),
  as.data.frame(posterior_draws$cloglog) %>%
    mutate(Iteration = row_number(), Model = "C-log-log")
) %>%
  pivot_longer(
    cols = c(beta0, beta_male, beta_age),
    names_to = "Parameter",
    values_to = "Value"
  )

## Plot trace
ggplot(trace_data, aes(x = Iteration, y = Value)) +
  geom_line(linewidth = 0.12) +
  facet_grid(Model ~ Parameter, scales = "free_y") +
  labs(x = "Iteration", y = "Parameter value") +
  theme_minimal()

# Posterior summaries (Table 1)
summarize_posterior <- function(samples, model_name) {
  data.frame(
    Model = model_name,

```

```

    Parameter = colnames(samples),
    Mean = apply(samples, 2, mean),
    Lower = apply(samples, 2, quantile, 0.025),
    Upper = apply(samples, 2, quantile, 0.975)
  )
}

table1 <- bind_rows(
  summarize_posterior(posterior_draws$probit, "Probit"),
  summarize_posterior(posterior_draws$logit, "Logit"),
  summarize_posterior(posterior_draws$cloglog, "C-log-log")
) %>%
  mutate(across(c(Mean, Lower, Upper), round, 3))

kable(
  table1,
  caption = "Posterior summaries for regression coefficients."
)

# Posterior Predictive Simulation
simulate_ppd <- function(samples, data, link) {
  total_survivors <- numeric(nrow(samples))

  for (s in 1:nrow(samples)) {
    beta <- samples[s, ]
    eta <- beta[1] + beta[2] * data$x_male + beta[3] * data$x_age

    if (link == "probit") {
      p <- pnorm(eta)
    } else if (link == "logit") {
      p <- plogis(eta)
    } else {
      p <- 1 - exp(-exp(eta))
    }

    y_rep <- rbinom(nrow(data), size = 1, prob = p)
    total_survivors[s] <- sum(y_rep)
  }

  total_survivors
}

## Generate posterior predictive samples
pp_probit <- simulate_ppd(posterior_draws$probit, dat, "probit")
pp_logit <- simulate_ppd(posterior_draws$logit, dat, "logit")
pp_cloglog <- simulate_ppd(posterior_draws$cloglog, dat, "cloglog")

```

```

# Posterior Predictive Table (Table 2)
pp_table <- data.frame(
  Model = c("Probit", "Logit", "C-log-log"),
  Mean = c(mean(pp_probit), mean(pp_logit), mean(pp_cloglog)),
  Lower = c(
    quantile(pp_probit, 0.025),
    quantile(pp_logit, 0.025),
    quantile(pp_cloglog, 0.025)
  ),
  Upper = c(
    quantile(pp_probit, 0.975),
    quantile(pp_logit, 0.975),
    quantile(pp_cloglog, 0.975)
  )
) %>%
  mutate(across(c(Mean, Lower, Upper), round, 2))

kable(
  pp_table,
  caption = "Posterior predictive distribution for the number of survivors."
)

```